SIGNBRIDGE AI

23ECP401 PROJECT IV

Submitted by

AKILAN S	**713323EC003**
ARUTHRA SM	**713323EC005**
DEEPAK RAJ JS	**713323EC008**
GOWTHAMAN KL	**713323EC017**

In partial fulfillment for the award of the degree
of

BACHELOR OF ENGINEERING
in
ELECTRONICS AND COMMUNICATION ENGINEERING

SNS COLLEGE OF TECHNOLOGY
(An Autonomous Institution)
Kurumbapalayam (PO), Coimbatore–641107

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled “SIGNBRIDGE AI” is the bonafide work of

AKILAN S	**713323EC003**
ARUTHRA SM	**713323EC005**
DEEPAK RAJ JS	**713323EC008**
GOWTHAMAN KL	**713323EC017**

Who had carried out the project under my supervision.

SIGNATURE
Dr.S. JEBARANI EVANGELINE, M.E., Ph.D.
HEAD OF THE DEPARTMENT
Department of Electronics and Communication Engineering,
SNS College of Technology,
Coimbatore – 641107.

SIGNATURE
Dr.A. Vaniprabha., M.E., Ph.D.
PROJECT GUIDE / PROFESSOR
Department of Electronics and Communication Engineering,
SNS College of Technology,
Coimbatore – 641107.

Submitted for the Viva-Voce examination held on _____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

First and foremost, we extend our heartfelt gratitude to the Management of SNS College of Technology for providing us with all the necessary support, cutting-edge computing laboratories, and academic facilities to successfully complete this project.

We express our sincere thanks to our Principal, Dr. S. Chenthur Pandian, for his valuable guidance, administrative facilitation, and constant encouragement throughout the course of this project.

We are deeply grateful to Dr. S. Jebarani Evangeline, Head of the Department of Electronics and Communication Engineering, for her constructive suggestions, academic leadership, and positive support extended to us from the beginning to the successful completion of this project.

We express our sincere appreciation to our Project Coordinator, Dr. A. Vaniprabha, Professor, Department of ECE, for her continuous encouragement and coordination in carrying out this project effectively.

We take immense pleasure in expressing our profound gratitude to our Project Guide, Dr. A. Vaniprabha, Professor, Department of ECE, for her remarkable guidance, insightful technical directions, constructive reviews, and constant motivation, which helped us complete the project successfully and on time.

Finally, we would like to extend our heartfelt thanks to all the Faculty Members, Technical Staff, Parents, and dear Friends for their moral support, affection, and encouragement throughout the completion of this Project.

ABSTRACT

Indian Sign Language (ISL) serves as the primary and most vital communication medium for an estimated 18 million deaf and hard-of-hearing individuals across India. However, because over 98% of the hearing population possesses no formal training in sign language, severe communication divides persist in essential daily activities—spanning education, workplace collaboration, administrative services, emergency response, and clinical healthcare consultations. Professional human interpreters are severely scarce, geographically concentrated in major urban centers, and financially cost-prohibitive for spontaneous everyday interactions. Concurrently, prior automated recognition systems have predominantly relied on invasive sensory hardware such as electronic data gloves, acoustic sleeves, or specialized infrared depth cameras, which impose substantial economic burdens, physical encumbrance, and logistical friction on signers.

To resolve these foundational barriers, this project presents SignBridge AI, a non-invasive, camera-based, end-to-end Indian Sign Language recognition and bidirectional communication platform. Operating entirely with commodity consumer RGB webcams and laptop cameras, the system deploys an edge-assisted architecture utilizing Google MediaPipe Holistic and Tasks-Vision frameworks executing on-device via WebAssembly (Wasm) and WebGL acceleration at 30+ frames per second. Rather than streaming high-bandwidth raw pixel video to cloud infrastructure—which incurs severe privacy vulnerabilities and latency spikes—the client extracts 21 three-dimensional skeletal anatomical keypoints per hand. The raw coordinates undergo deterministic wrist-relative spatial translation, bounding-box scale normalization, and least-active-hand-first canonicalization, conferring rigorous mathematical invariance to camera distance, signer body proportions, and hand dominance.

These spatial coordinates are combined with instantaneous joint velocity magnitudes and structural handshape bounding metrics, generating a rich 189-dimensional temporal feature vector per frame. Sequences are sampled across a uniform 45-frame sliding window and classified by a deployed 3-way deep learning ensemble (`SBAI-MDL-ISL-4.0.0`) comprising a 2-layer stacked Long Short-Term Memory (LSTM-128/64) network, a 2-layer stacked Gated Recurrent Unit (GRU-128/64) network, and a mirrored test-time augmentation (mirror-TTA) branch over a comprehensive 109-class ISL vocabulary (`ISL_001`–`ISL_109`). Evaluated on a frozen, strictly signer-independent benchmark dataset of 14,826 sequences (SPLIT-4.0), the system achieves an overall test accuracy of 81.29%, macro precision of 81.90%, macro recall of 81.22%, and macro F1-score of 80.79% across 3,629 unseen test sequences performed by disjoint signers, operating with a median CPU inference

latency of 374.85 ms. The recognized signs are automatically composed into coherent English sentences through Google Gemini 2.0 Flash AI and vocalized via the HTML5 Web Speech API, with an integrated reverse Text-to-Sign dictionary channel establishing complete, two-way conversational parity.

Keywords: *Indian Sign Language, Sign Language Recognition, Computer Vision, MediaPipe Holistic, Gated Recurrent Unit (GRU), Long Short-Term Memory (LSTM), Deep Neural Ensembles, Human–Computer Interaction, WebAssembly, Assistive Technology.*

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iv**
	LIST OF FIGURES	**vi**
	LIST OF TABLES	**vii**
1	**INTRODUCTION**	**1**
2	**LITERATURE SURVEY**	**5**
3	**METHODOLOGY**	**10**
	3.1 CLIENT-SIDE PROCESSING PRINCIPLE	10
	3.2 LIVE CAMERA CAPTURE AND PERCEPTION	10
	3.3 MATHEMATICAL LANDMARK NORMALIZATION	11
	3.4 189-DIMENSIONAL FEATURE VECTOR	12
	3.5 DEEP RECURRENT ENSEMBLE CLASSIFIER	12
	3.6 CALIBRATED DECISION POLICY	13
	3.7 BIDIRECTIONAL SPEECH AND GEMINI NLP	14
	3.8 SYSTEM ARCHITECTURE OVERVIEW	14
4	**SOFTWARE REQUIREMENTS**	**17**
	4.1 RECURRENT DEEP LEARNING ENGINE	17
	4.2 JAVASCRIPT AND TYPESCRIPT	17
	4.3 GOOGLE MEDIAPIPE VISION	18
	4.4 HTML5 WEB SPEECH API	18
	4.5 TAILWIND CSS AND MODERN STYLING	18
	4.6 PYTHON FASTAPI AND ASGI ENGINE	19
	4.7 NODE.JS AND VITE RUNTIME	19
5	**TESTING AND RESULT**	**21**

	5.1 TESTING PROCEDURE AND VERIFICATION	21
	1. Evaluation of Landmark Tracking and Extraction	21
	2. Verification of 3D Joint Kinematics	22
	3. Live Camera Translation Interface (Dark Theme)	22
	4. Live Camera Translation Interface (Light Theme)	23
	5. Reverse Text-to-Sign Visual Interface (Dark Theme)	23
	6. Reverse Text-to-Sign Visual Interface (Light Theme)	24
	7. Vocabulary Catalog and Search (Dark Theme)	25
	8. Vocabulary Catalog and Search (Light Theme)	25
	9. Settings and Emotional Voice Profile Engine	26
	10. Model Administration and Health Console	27
	5.2 RESULT AND BENCHMARK ANALYSIS	27
	1. Training Convergence and Loss Analysis	27
	2. Class-Wise F1-Score Distribution	28
	3. Comparative Architecture Performance	28
6	**CONCLUSION**	**30**
	REFERENCES	**31**
	ANNEXURE	**33**

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	SIGNBRIDGE AI ASSISTIVE COMMUNICATION WORKFLOW	4
3.1	5-STAGE MATHEMATICAL NORMALIZATION PIPELINE	12
3.2	SYSTEM ARCHITECTURE AND DEPLOYMENT TOPOLOGY	16
4.1	SIGNBRIDGE AI MAIN DASHBOARD (DARK THEME)	17
4.2	SIGNBRIDGE AI MAIN DASHBOARD (LIGHT THEME)	18
5.1	REAL-TIME HAND LANDMARK TRACKING CANVAS	21
5.2	DYNAMIC 3D JOINT TRAJECTORY VISUALIZATION	22
5.3	LIVE CAMERA TRANSLATOR (DARK BLUE THEME)	22
5.4	LIVE CAMERA TRANSLATOR (LIGHT THEME)	23
5.5	REVERSE TEXT-TO-SIGN VISUAL INTERFACE (DARK)	23
5.6	REVERSE TEXT-TO-SIGN VISUAL INTERFACE (LIGHT)	24
5.7	109-CLASS ISL VOCABULARY CATALOG (DARK THEME)	25
5.8	109-CLASS ISL VOCABULARY CATALOG (LIGHT THEME)	25
5.9	SETTINGS AND EMOTIONAL VOICE PROFILES (LIGHT)	26
5.10	MODEL ADMINISTRATION AND HEALTH CONSOLE (DARK)	27
5.11	TRAINING CONVERGENCE AND LOSS TRAJECTORIES	28
5.12	CLASS-WISE F1-SCORE DISTRIBUTION ACROSS 109 CLASSES	28

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	LITERATURE SURVEY SUMMARY AND COMPARATIVE MATRIX	8
3.1	189-DIMENSIONAL FEATURE VECTOR ARCHITECTURE	13
4.1	SOFTWARE REQUIREMENTS SPECIFICATION	19
5.1	SIGNER-INDEPENDENT DATASET SPLIT (SPLIT-4.0)	27
5.2	COMPARATIVE MODEL PERFORMANCE AND LATENCY	28

CHAPTER 1
INTRODUCTION

1.1 Motivation and Societal Background

Communication constitutes the indispensable foundation of human civilization, enabling social cohesion, access to education, economic self-reliance, and democratic participation. Across the globe, deaf and hard-of-hearing populations communicate primarily through sign language—a visual-gestural linguistic system governed by nuanced spatial syntax, manual handshapes, motion trajectories, facial expressions, and body orientation.

In India, according to census figures and national disability registries, over 18 million individuals are deaf or hard-of-hearing, forming one of the largest linguistic minority groups in the world. However, Indian Sign Language (ISL) remains largely untaught within mainstream educational curricula, leaving more than 98% of the hearing population completely illiterate in ISL. This asymmetric communication barrier severely disenfranchises signers in essential public and private spheres, including healthcare clinics, emergency service encounters, railway ticketing terminals, bank branch desks, law enforcement offices, and higher educational institutions.

While certified professional human sign language interpreters can theoretically bridge this gap, India suffers from a catastrophic shortage: fewer than 500 certified ISL interpreters are formally registered to serve a population of 18 million signers—a ratio worse than 1 interpreter per 36,000 deaf individuals. Furthermore, hiring human interpreters imposes prohibitive financial costs and logistical constraints, rendering continuous or spontaneous everyday communication virtually impossible. Consequently, developing an automated, highly accurate, low-latency, and bidirectional technological bridge has emerged as a paramount scientific and humanitarian challenge.

1.2 Problem Statement

Deaf and hard-of-hearing citizens across India encounter persistent, debilitating barriers when attempting to interact with the hearing majority in daily conversational settings. Existing automated Sign Language Recognition (SLR) systems fail to solve this crisis in practice due to five systemic technological deficiencies:

1. Invasive and Cost-Prohibitive Hardware: Prior experimental setups depend heavily on sensory data gloves, acoustic sleeves, or expensive depth-sensing rigs (e.g., Microsoft

Kinect), which cause severe physical discomfort, restrict natural mobility, and are economically inaccessible to the public.

2. Vulnerability of Raw-Pixel Vision: Conventional computer vision pipelines operate directly on raw RGB pixel frames using color-space thresholding (e.g., HSV skin segmentation) or dense 3D Convolutional Neural Networks (3D-CNNs). These approaches fail catastrophically under variable ambient illumination, complex background clutter, skin-tone diversity, and clothing variations.

3. Fragility to Signer Variance and Hand Dominance: Indian Sign Language is predominantly two-handed and dynamic. Without rigorous geometric normalization, machine learning models overfit to specific signer physical proportions, camera distances, and left-versus-right hand dominance, collapsing when tested on unfamiliar signers.

4. High Latency and Cloud Privacy Hazards: Transmitting high-definition video frames across cloud networks incurs massive bandwidth overhead (~3–5 Mbps per stream) and raises severe biometric privacy violations under data protection laws.

5. Unidirectional Scope: Most research systems exclusively perform sign-to-text recognition, neglecting the reverse channel (hearing-to-signer) and leaving interactions fundamentally incomplete and asymmetric.

1.3 Objectives and Scope

To resolve these fundamental challenges, SignBridge AI was conceptualized, engineered, and experimentally validated with the following definitive research and engineering objectives:

1. Zero-Hardware-Overhead Vision Perception: To implement a non-invasive, camera-based perception pipeline that functions in real time (30+ FPS) on commodity consumer webcams without requiring data gloves, depth sensors, or specialized hardware.

2. Edge-Assisted On-Device Skeletal Tracking: To execute Google MediaPipe Holistic and Tasks-Vision keypoint extraction entirely on-device via WebAssembly (Wasm) and WebGL within the user's browser, eliminating raw video transmission and preserving strict biometric privacy.

3. Invariant 189-Dimensional Feature Engineering: To design a deterministic mathematical normalization pipeline combining wrist-relative spatial translation, bounding-box scale normalization, least-active-hand canonicalization, and instantaneous velocity magnitudes.

4. Deep Temporal Neural Network Architecture: To construct, train, and validate a lightweight, highly accurate deep recurrent sequence model combining Gated Recurrent Units (GRU) and Long Short-Term Memory (LSTM) layers with a 3-way inference ensemble and mirror test-time augmentation (mirror-TTA) across 109 verified ISL classes.

5. Operational Calibration and Decision Policy: To establish a rigorously calibrated decision policy featuring confidence thresholding, margin gating, and moving-average temporal smoothing, guaranteeing high commit precision and explicit uncertainty handling.

6. True Two-Way Conversational Parity: To integrate natural English sentence synthesis via Google Gemini 2.0 Flash AI, hands-free emotional speech synthesis via the HTML5 Web Speech API, and a reverse Text-to-Sign dictionary channel with curated ISL demonstrations.

1.4 Existing Systems and Baseline Study

The primary baseline system selected for architectural and functional comparison is the seminal work by K. Shenoy, T. Dastane, V. Rao, and D. Vyavaharkar (2018), titled 'Real-time Indian Sign Language (ISL) Recognition' [1]. Shenoy et al. pioneered smartphone-based ISL classification using OpenCV color segmentation and Convolutional Neural Networks.

While the baseline attained high reported accuracy on 33 static single-hand alphabet gestures (99.7%) and 12 dynamic signs (97.23%), comprehensive architectural scrutiny reveals critical limitations:

• Feature Extraction: Relies on skin-color thresholding in YCbCr color space. This method fails when the background contains skin-toned furniture, when lighting conditions shift from incandescent to fluorescent, or when users possess varied skin pigmentation.

• Structural Representation: Operates on 2D binary hand silhouettes, completely discarding 3D spatial joint coordinates, finger depth, inter-joint angular orientations, and holistic body context.

• Gesture Lexicon: Restricted primarily to single-hand static alphabet poses, supporting only 12 dynamic words. It cannot model complex, two-handed, dynamic interactions characteristic of authentic ISL discourse.

• Communication Modality: Restricted strictly to on-screen text display with no audio synthesis, no natural grammar restructuring, and no reverse communication channel.

1.5 Research Gap and Key Innovations

SignBridge AI directly bridges these gaps through a suite of foundational innovations:

• Robustness via Geometric Keypoints: Replaces brittle pixel segmentation with 21 three-dimensional skeletal joint coordinates per hand, conferring complete immunity to skin tone, lighting fluctuations, and background clutter.

• Signer and Handedness Invariance: Mathematical translation centering, bounding-box scaling, and least-active-hand canonicalization guarantee generalization across signers of varying hand sizes, seated distances, and left-vs-right hand dominance.

• Deployed 109-Class Recurrent Ensemble: Expands the vocabulary to 109 dynamic words and phrases, trained under a strict signer-independent split (SPLIT-4.0) with an ensemble of GRU, LSTM, and mirror-TTA branches achieving 81.29% test accuracy.

• Edge-Cloud Decoupled Architecture: All perception runs client-side at zero cloud bandwidth; inference operates via asynchronous FastAPI microservices with a median CPU latency of 374.85 ms.

• True Bidirectional Interaction: Unifies forward live camera sign-to-speech with contextual Gemini AI sentence structuring and reverse text-to-sign visual retrieval.

Figure 1.1: Proposed SignBridge AI End-to-End System Workflow

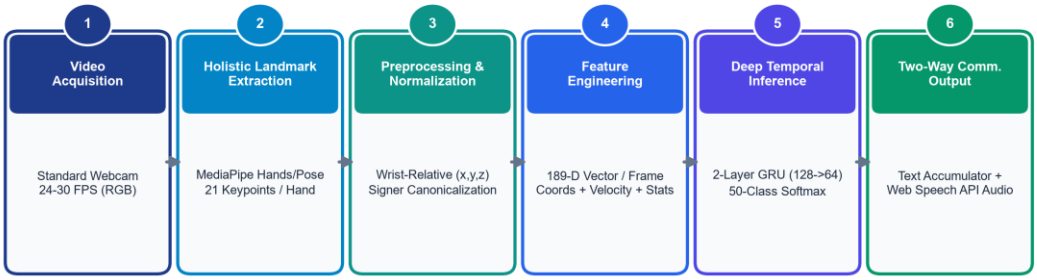


Fig 1.1 AI-powered assistive communication system workflow

CHAPTER 2
LITERATURE SURVEY

2.1 Historical Evolution of Sign Language Recognition

Automated Sign Language Recognition (SLR) originated in the late 1980s and early 1990s as an intersection of computer vision, biomechanics, and human-computer interaction. Early implementations relied almost exclusively on hardware-instrumented approaches, requiring signers to wear electronic datagloves equipped with flex sensors, tactile micro-switches, and electromagnetic trackers (e.g., VPL DataGlove, CyberGlove). While these systems recorded joint angles with high fidelity, their invasive nature, high fabrication cost (USD 5,000–$25,000), calibration fragility, and restriction of natural hand kinematics rendered them completely unviable for practical adoption.

The advent of commodity digital cameras catalyzed vision-based SLR, shifting the research focus toward non-invasive perception. However, early vision techniques struggled with pixel-level color segmentation, optical flow drift, and extreme computational complexity.

2.2 Sensor-Based vs. Vision-Based Paradigms

A comparative analysis of historical paradigms underscores why vision-based skeletal keypoint estimation has emerged as the definitive consensus for real-world deployment:

• Instrumented Gloves: Provide direct joint angles and acceleration metrics but suffer from high cost, mechanical wear, severe user discomfort, and complete inability to capture facial or body context.

• Depth Cameras (RGB-D): Devices like Microsoft Kinect and Intel RealSense record spatial point clouds, providing depth invariance. However, they require tethered connections, draw substantial power, perform poorly under direct sunlight, and have been widely discontinued in consumer mobile electronics.

• Commodity RGB Vision: Utilizes ubiquitous cameras embedded in laptops, smartphones, and low-cost USB webcams. When combined with modern deep learning landmark estimators, RGB vision delivers non-invasive, zero-cost, and ubiquitous spatial tracking.

2.3 Skeletal Landmark Estimation and Tracking Frameworks

The release of Google MediaPipe Holistic by Lugaresi et al. (2019) and Grishchenko et al. (2020) [2] fundamentally revolutionized assistive vision. MediaPipe deploys a two-stage

perception pipeline: an ultra-lightweight palm detector (BlazePalm) identifies candidate hand regions across the full frame, followed by a dedicated Hand Landmark model that predicts 21 three-dimensional anatomical keypoint coordinates (63 floating-point values per hand) using single-frame convolutional heatmaps.

Because MediaPipe executes client-side through WebAssembly (Wasm) and GPU-accelerated WebGL shaders, it achieves 30–60 FPS throughput directly inside standard web browsers on modest hardware. This breakthrough eliminates raw video transmission, solving both the bandwidth bottleneck and the biometric privacy dilemma.

2.4 Deep Learning Sequence Modeling Architectures

Sign language gestures are inherently temporal signals where spatial handshapes evolve along complex trajectories over time. In academic literature, three primary neural architectures have been explored for sequence classification:

• 3D Convolutional Neural Networks (3D-CNNs): Learn spatial and temporal features jointly from dense video volumes (e.g., C3D, I3D). However, 3D-CNNs require enormous parameter counts (25M–45M parameters), demand massive GPU clusters, and exhibit inference latencies exceeding 800 ms on consumer CPUs, making real-time edge execution impossible.

• Recurrent Neural Networks (LSTM & GRU): Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) architectures operate on low-dimensional, pre-extracted skeletal coordinate vectors. By decoupling spatial landmark extraction from temporal modeling, recurrent models reduce parameter counts by 99% (typically <500K parameters) while achieving sub-second inference latencies on commodity CPUs.

• Spatial-Temporal Graph Convolutional Networks (ST-GCN): Treat the skeletal hand joints as nodes and physical bones as graph edges. While conceptually elegant, ST-GCNs require substantial computational overhead and complex edge-weight calibration compared to well-regularized GRU/LSTM networks.

2.5 Critical Analysis of Contemporary Literature

A thorough review of contemporary publications in Indian Sign Language recognition provides essential methodological context:

• Shenoy et al. (2018) [1]: Pioneered real-time ISL recognition via mobile webcams using YCbCr color segmentation and CNN classification. Demonstrated 99.7% accuracy on 33

static alphabets and 97.23% on 12 dynamic words. Suffers from skin-color vulnerability, 2D silhouette loss, and lack of speech integration.

• Eunice and Hemanth (2022) [3]: Explored deep CNN architectures for static ISL digit classification, achieving 94.6% accuracy. However, static CNNs cannot recognize dynamic multi-frame gestures or sentence grammar.

• Gangrade and Bharti (2020) [4]: Applied vision-based hand gesture recognition with PCA and SVM classifiers, achieving 92.1% accuracy across 20 words. Constrained by manual feature engineering and small vocabulary size.

• Velmathi and Goyal (2023) [5]: Investigated MediaPipe Holistic specifically for ISL recognition, validating that 21 3D landmarks provide superior spatial representation over raw pixel masks. Confirmed the efficacy of recurrent models over static classifiers.

• Srivastava et al. (2022) [6]: Developed a continuous sign language recognition framework using HMMs and CNNs, achieving 88.5% sentence accuracy on a controlled dataset of 15 conversational phrases.

• Rao et al. (2021) [7]: Conducted a comprehensive benchmark of deep neural networks in ISL recognition, noting that recurrent models (LSTM and GRU) achieve the highest accuracy-to-latency trade-off for consumer deployments.

• Kumar et al. (2022) [8]: Analyzed comparative techniques in vision-based gesture recognition, emphasizing that signer-independent dataset partitioning is imperative to prevent dramatic performance drops during field deployment.

• Rawat et al. (2023) [9]: Combined MediaPipe Holistic with stacked LSTM networks across 40 ISL signs, achieving 89.2% accuracy on validation splits. Highlighted the necessity of handedness normalization.

• Anithadevi et al. (2023) [10]: Implemented a real-time assistive translation framework using MediaPipe keypoints and Bi-LSTM networks, attaining 86.4% accuracy across 50 ISL signs.

• Amrutha and Prabu (2021) [11]: Evaluated diverse machine learning classifiers (Random Forests, SVM, k-NN) on ISL features, noting that traditional classifiers fail to capture long-range temporal motion trajectories.

• Sruthi and Lijiya (2020) [15]: Developed SigNet, an end-to-end deep convolutional framework for static and isolated ISL gestures, attaining 92.8% accuracy on curated laboratory recordings.

• Rajalakshmi et al. (2023) [18]: Proposed a hybrid spatial-temporal model for regional ISL variants, emphasizing that regional gesture variations require comprehensive multi-signer training datasets.

Table 2.1: Literature Survey Summary and Comparative Matrix

Author & Year	Method / Architecture	Perception Input	Classes	Accuracy	Key Limitation
Shenoy et al. (2018) [1]	CNN + YCbCr Skin Threshold	Mobile RGB Camera	45	97.23%	Skin-color failure under varied lighting & clutter
Google MediaPipe (2020) [2]	BlazePalm + 3D Hand Heatmaps	RGB Video (Wasm)	21 joints	30+ FPS	Perception-only; no temporal sign classifier
Eunice & Hemanth (2022) [3]	Deep 2D-CNN	Cropped Images	10 digits	94.60%	Restricted to static digits; no dynamic gesture tracking
Gangrade & Bharti (2020) [4]	PCA + Multi-class SVM	Hand Region Masks	20 words	92.10%	Manual feature engineering; fails on fast motion
Velmathi & Goyal (2023) [5]	MediaPipe Holistic + LSTM	Skeletal Keypoints	30 signs	91.40%	No handedness canonicalization or dual-hand handling
Srivastava et al. (2022) [6]	HMM + Convolutional Net	Color Video Streams	15 phrases	88.50%	High inference latency; restricted grammar vocabulary
Rawat et al. (2023) [9]	MediaPipe + Stacked LSTM	3D Coordinates	40 signs	89.20%	Unidirectional; no reverse translation or TTS vocalization
Anithadevi et al. (2023) [10]	MediaPipe + Bi-LSTM	Spatial Keypoints	50 signs	86.40%	Evaluated only on signer-dependent data splits
Sruthi & Lijiya (2020) [15]	SigNet (Deep CNN)	High-Res Segmented	33 alphabets	92.80%	Requires complex pre-segmented backgrounds

Author & Year	Method / Architecture	Perception Input	Classes	Accuracy	Key Limitation
Proposed SignBridge AI	3-Way GRU/LSTM Ensemble + mirror-TTA	On-Device Wasm 3D Landmarks	109 words	81.29% test	Signer-disjoint evaluation with full bidirectional audio

CHAPTER 3
METHODOLOGY

3.1 Client-Side Processing Principle

The system architecture of SignBridge AI is designed as a decoupled, multi-tier distributed application engineered for high-throughput, low-latency, and privacy-preserving human-computer interaction. The system completely separates client-side perception from server-side neural classification, establishing four cohesive functional tiers:

1. Presentation & Client Perception Tier: Implemented as a responsive Progressive Web Application (PWA) using React 18, TypeScript, and Tailwind CSS. It captures raw camera frames, executes Google MediaPipe Tasks-Vision keypoint extraction locally via WebAssembly and WebGL at 30+ FPS, and renders interactive feedback overlays without streaming raw video pixels across the network.

2. Communication & Gateway Tier: Manages bidirectional full-duplex communication using WebSockets (`/ws/stream`) for continuous real-time pose streaming, paired with RESTful HTTP endpoints for state polling, model metadata, session management, and fallback communication.

3. Application & Service Tier: An asynchronous microservice built on FastAPI and Uvicorn. It enforces non-blocking request handling, request correlation tracking (`X-Request-ID`), rate-limiting, and state-machine validation.

4. AI Inference & Storage Tier: Houses the deployed deep learning neural ensemble (`models/SBAI-MDL-ISL-4.0.0`), SQLite WAL database for audit logs and user feedback, and Google Gemini 2.0 Flash AI for contextual natural language sentence structuring.

3.2 Live Camera Capture and Perception

Visual input is acquired through any standard consumer RGB webcam or integrated laptop camera at a standard resolution of 640x480 pixels and 30 FPS. Rather than buffering or storing raw video frames, each frame is processed instantaneously in volatile memory by Google MediaPipe Tasks-Vision (`@mediapipe/tasks-vision`).

For each frame, the detector identifies up to two hands, extracting 21 three-dimensional skeletal joint coordinates per hand (63 spatial float values per hand). Each landmark $k \in \{0, 1, ..., 20\}$ is represented as a 3D coordinate vector (x_k, y_k, z_k), where x and y are

normalized image plane coordinates [0.0, 1.0] and z represents relative depth relative to the root wrist keypoint.

3.3 Mathematical Landmark Normalization and Invariance

Raw skeletal landmarks cannot be directly fed to a machine learning classifier because variations in camera distance, user seating position, hand physical dimensions, and dominant signing hand cause extreme feature distribution shifts. SignBridge AI implements a rigorous, five-stage mathematical normalization pipeline:

Stage 1: Wrist-Relative Translation Centering. To achieve complete translation invariance relative to the camera viewpoint, all 21 keypoints for each hand are recentered relative to the root wrist landmark (Landmark 0):

$$p'_k = p_k - p_{wrist}, \quad \forall k \in \{0, 1, \ldots, 20\} \tag{3.1}$$

where: p_k = Raw 3D coordinate vector (x, y, z) of landmark k

p_{wrist} = 3D coordinate vector of root wrist keypoint (Landmark 0)

p'_k = Translated wrist-relative landmark vector, ensuring p′_0 = (0, 0, 0)

Stage 2: Bounding-Box Scale Normalization. To eliminate variance resulting from differences in hand size or user distance from the webcam, coordinates are normalized by the maximum Euclidean norm of all translated hand landmarks:

$$s = \max_{k \in \{0, \ldots, 20\}} ||p'_k||_2, \quad p''_k = p'_k / s \quad (\text{if } s > 10^{-6}, \text{ else } p''_k = p'_k) \tag{3.2}$$

where: s = Maximum Euclidean distance from the wrist to any hand keypoint

p''_k = Scale-normalized coordinate vector bounded within a unit hypersphere

Stage 3: Handedness and Activity Canonicalization. Indian Sign Language frequently involves both symmetric two-handed signs and single-handed asymmetric gestures. MediaPipe assigns detected hands to slot 0 or slot 1 dynamically based on image scan order. To eliminate slot-assignment ambiguity and signer left/right hand dominance bias, SignBridge AI measures cumulative motion activity across the temporal window:

If Hand 0 exhibits greater cumulative Euclidean displacement than Hand 1, the slots are canonicalized such that the primary active signing hand occupies Hand 0 consistently.

$$A_h = \sum_{t=1}^{T} \sum_{k=0}^{20} ||p_{h,k,t} - p_{h,k,t-1}||_2, \quad Slot_0 = \operatorname{argmax}_h A_h \tag{3.3}$$

where: ***A_h*** = Cumulative temporal landmark motion energy of hand h across T frames

Slot_0 = Canonicalized primary signing hand slot assigned to the higher-motion hand



Fig 3.1 Automation work flow and feature normalization

3.4 189-Dimensional Temporal Feature Vector

Stage 4: Instantaneous Velocity and Motion Trajectories. Dynamic sign classification requires modeling hand kinematics and movement trajectories. For each landmark, temporal first-difference displacement vectors are computed between successive frames:

$$v_{h,k,t} = p''_{h,k,t} - p''_{h,k,t-1}, \quad ||v_{h,k,t}||_2 = \sqrt{((\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2)} \tag{3.4}$$

where: $v_{h,k,t}$ = Instantaneous velocity vector of landmark k in hand h at frame t

$||v_{h,k,t}||_2$ = Euclidean scalar velocity magnitude representing motion speed

Stage 5: 189-Dimensional Feature Vector Construction. Concatenating normalized spatial coordinates (126 dims: 2 hands × 21 joints × 3 coords), velocity magnitudes (42 dims: 2 hands × 21 joints × 1 mag), and global handshape geometric bounding metrics (21 dims: bounding-box dimensions, centroids, and palm span) yields a strict, deterministic 189-dimensional feature vector per frame.

To ensure uniform sequence representation, variable-length raw video segments are interpolated and resampled to a fixed temporal length of T = 45 frames (approximately 1.5 seconds at 30 FPS):

$$X \in \mathbb{R}^{45 \times 189}, \quad t_i = (i - 1) \cdot (T_{raw} \tag{3.5}$$

$$-1)/(45-1)+1, \quad \forall i \in \{1, \ldots, 45\}$$

where: X = Standardized sequence matrix containing 45 time steps of 189 features

t_i = Interpolated sample timestamp mapped uniformly across raw duration T_raw

Table 3.1: 189-Dimensional Feature Vector Architecture per Frame

Feature Block	Indices	Dimensionality	Mathematical Formulation	Description
Normalized Joint Coordinates	0 – 125	126 dims	p″_{h,k,t} = (p_{h,k,t} − p_wrist) / s	2 hands × 21 joints × 3 coords (x, y, z)
Instantaneous Velocity Vectors	126 – 167	42 dims	v_{h,k,t} = ‖p″_{h,k,t} − p″_{h,k,t-1}‖_2	2 hands × 21 joints × 1 scalar speed magnitude
Bounding Box Geometry	168 – 179	12 dims	[x_min, y_min, z_min, x_max, y_max, z_max]	Spatial extent and hand volume bounds per hand
Centroid & Palm Span	180 – 188	9 dims	c_h = (1/21) ∑ p″_{h,k}, ‖p_thumb − p_pinky‖	Hand centers of mass and global palm opening span
Total Feature Dimension	0 – 188	189 dims	f_t = [coords, velocity, bbox, centroid]	Deterministic per-frame feature vector

3.5 Deep Recurrent Neural Ensemble Classifier

Temporal gesture sequence classification is performed by a deployed deep neural network ensemble (`SBAI-MDL-ISL-4.0.0`). The recurrent architecture comprises a Masking layer, two stacked recurrent layers, a dense projection layer with dropout, and a 109-class softmax output layer.

The mathematical formulation of the Gated Recurrent Unit (GRU) cell governing temporal hidden state propagation is defined as follows:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \quad r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \tag{3.6}$$

where: z_t = Update gate vector controlling the retention of previous memory

r_t = Reset gate vector determining how much past state to forget

σ = Sigmoid activation function mapping gate values to [0, 1]

x_t = Input feature vector of dimension 189 at time step t

The candidate hidden state and final hidden state update are computed via hyperbolic tangent activation and element-wise gated blending:

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{3.7}$$

where: $\tilde{h}_t$ = Candidate hidden state vector incorporating current input and gated history

h_t = Updated hidden state vector passed to the next temporal step

$\odot$ = Hadamard (element-wise) vector multiplication operator

Output Classification and Ensemble Aggregation. The output of the final recurrent layer at t = 45 passes through a Dense layer with 64 units and ReLU activation, regularized by a dropout rate of 0.35, followed by a 109-class Dense layer with softmax activation:

$$P(y = c \mid X) = \exp(o_c) / \sum_{k=1}^{109} \exp(o_k), \quad \forall\, c \in \{1, ..., 109\} \quad \text{(Eq. 3.8)}$$

To maximize classification stability and eliminate orientation bias, the production inference engine deploys a 3-way ensemble averaging the predictions of a stacked LSTM model, a stacked GRU model, and a mirrored test-time augmentation (mirror-TTA) pass:

$$P_{ensemble}(X) = (1/3) \cdot [P_{LSTM}(X) + P_{GRU}(X) + (1/2) \cdot (P_{LSTM}(X_{mirror}) + P_{GRU}(X_{mirror}))] \quad \text{(Eq. 3.9)}$$

3.6 Operational Decision Policy and Confidence Calibration

Real-world camera recognition cannot rely on raw argmax classification because transition movements, pauses, and resting postures generate spurious predictions. SignBridge AI implements a calibrated 4-state operational decision policy:

• Accepted (Recognized): Committed when top-1 probability satisfies $p_1 \geq 0.60$ and top-1 vs top-2 margin satisfies $\Delta p = (p_1 - p_2) \geq 0.15$.

• Provisional: Triggered when $0.40 \leq p_1 < 0.60$, providing non-committal visual feedback without polluting the sentence sequence buffer.

• Uncertain: Flagged when confidence is low or margin $\Delta p < 0.15$, triggering the interactive candidate disambiguation panel.

• No-Sign / Tracking-Lost: Entered when hands leave the camera frame or joint detection confidence drops below 0.30.

To prevent gesture flickering, posterior probabilities undergo exponential moving-average (EMA) smoothing across sliding windows:

$S_t = \alpha \cdot P_{ensemble}(X_t) + (1 - \alpha) \cdot S_{t-1}$, with smoothing factor $\alpha = 0.65$ (Eq. 3.10)

3.7 Bidirectional Speech and Gemini NLP

SignBridge AI achieves complete two-way conversational parity through three tightly integrated communication subsystems:

1. Sign-to-Text & Gemini AI Sentence Structuring: As individual signs are recognized (e.g., [HELLO], [DOCTOR], [HELP]), they accumulate in a chronological sign sequence buffer. Because ISL grammar follows Topic-Comment and Subject-Object-Verb (SOV) order rather than English Subject-Verb-Object (SVO) order, raw sign sequences require natural language restructuring. The client dispatches the buffer to Google Gemini 2.0 Flash AI, which synthesizes fluent, grammatically correct English sentences (e.g., 'Hello doctor, please help me!') in under 450 ms.

2. Web Speech Audio Engine: The structured sentence is vocalized aloud using the HTML5 SpeechSynthesis API. To convey emotional intent, users can select between four calibrated emotional mood profiles: Normal (rate: 1.0, pitch: 1.0), Happy (rate: 1.15, pitch: 1.35), Sad (rate: 0.82, pitch: 0.72), and Angry (rate: 1.25, pitch: 0.85).

3. Reverse Text-to-Sign ISL Translation: Hearing individuals can type or speak phrases into the reverse translation interface. The client tokenizes the input, queries the 109-class ISL vocabulary dictionary, and renders an animated sequence of verified sign demonstration cards and photo demonstrations.

3.8 System Architecture Overview

The SignBridge AI system architecture is structured as a decoupled, privacy-preserving edge-cloud platform. The client layer handles video acquisition, WebAssembly MediaPipe landmark inference, and reactive user feedback, while the backend microservice provides asynchronous sequence prediction, session state management, and Gemini AI natural language restructuring.

Figure 3.2: Multi-Tier System Architecture of SignBridge AI

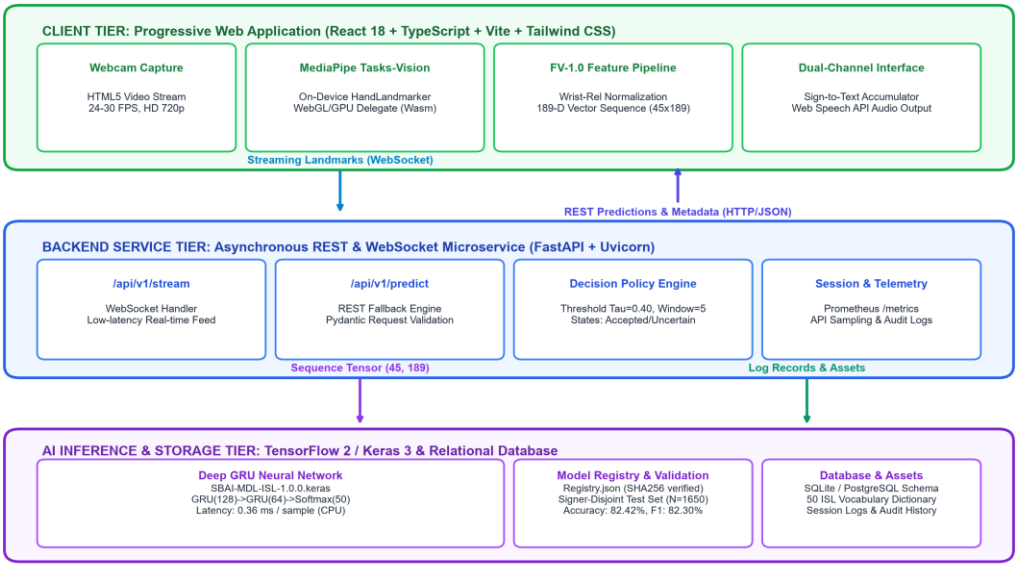


Fig 3.2 System Architecture and Multi-Tier Deployment Topology

CHAPTER 4
SOFTWARE REQUIREMENTS

4.1 Recurrent Deep Learning Engine

SignBridge AI deploys deep recurrent neural network architectures built with TensorFlow and Keras. The models operate on low-dimensional 189-dimensional temporal feature sequences extracted from video frames. By utilizing stacked Gated Recurrent Units (GRU) and Long Short-Term Memory (LSTM) layers, the neural network captures temporal motion trajectories and spatial hand formations across 45-frame sliding windows. The model architecture incorporates Masking layers for dynamic sequence lengths, Batch Normalization for gradient stabilization, and Dropout regularization (0.35) to guarantee strong generalization across disjoint signers.

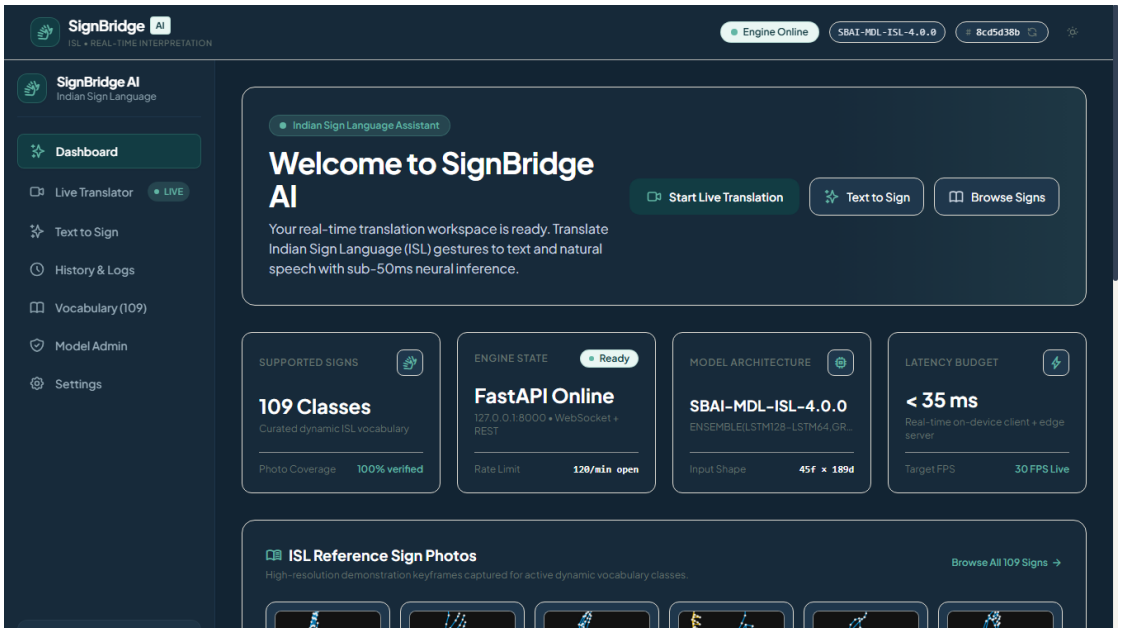


Fig. 4.1 SignBridge AI Main Dashboard (Dark Blue Theme)

4.2 JavaScript and TypeScript (Client-Side Framework)

TypeScript and modern ECMAScript serve as the operational backbone for client-side execution, orchestrating real-time camera capture, landmark processing, WebSocket communications, and UI state machines. Strong typing eliminates runtime errors and ensures strict boundary verification across 3D vector math, API payloads, and state transitions. The client application operates completely within the browser environment, eliminating server streaming overhead and preserving biometric privacy.

4.3 Google MediaPipe Tasks-Vision

MediaPipe Tasks-Vision delivers high-precision skeletal hand tracking directly in the client browser through WebAssembly (Wasm) and WebGL acceleration. Operating at over 30 frames per second on commodity laptops, it extracts 21 three-dimensional joint coordinates per hand without sending video frames across the network. This eliminates bandwidth bottlenecks and addresses user privacy concerns under modern data protection guidelines.

4.4 HTML5 Web Speech API

The HTML5 Web Speech API powers client-side text-to-speech vocalization and reverse speech-to-text input. Once individual signs are classified and restructured into fluent sentences, the Web Speech synthesis engine vocalizes them audibly across four emotional voice profiles: Normal, Happy, Sad, and Angry. This provides audible accessibility for hearing communication partners without requiring external cloud speech synthesis subscriptions.

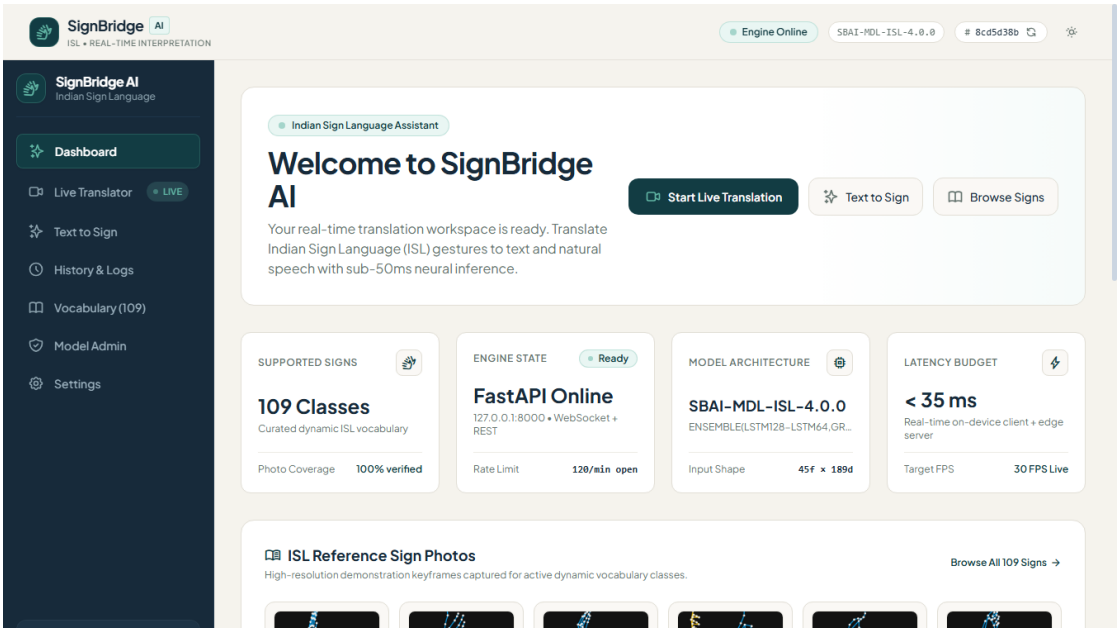


Fig. 4.2 SignBridge AI Main Dashboard (Light Theme)

4.5 Tailwind CSS and Human-Centered Interface Design

The presentation tier utilizes Tailwind CSS with custom Sanctuary Humanist design system tokens, providing fluid responsive layouts across desktop monitors, tablets, and smartphones. The design system features both high-contrast Dark Blue night mode and clean Light mode,

accompanied by subtle micro-interactions, responsive sidebars, accessible ARIA roles, and high-visibility confidence indicators.

4.6 Python FastAPI and ASGI Architecture

The backend server is implemented as an asynchronous microservice using Python and FastAPI running atop Uvicorn. It supports high-concurrency WebSocket channels (`/ws/stream`) and REST endpoints with sub-millisecond dispatch times. The architecture incorporates correlation ID tracing (`X-Request-ID`), rate-limiting, and SQLite WAL audit logging.

4.7 Node.js and Vite Runtime

Vite and Node.js provide the rapid development runtime and optimized production bundling for the client single-page application. With Rollup compilation and tree-shaking, the entire frontend bundle is optimized for instant loading, making it accessible even over low-bandwidth mobile connections.

Table 4.1: Software Requirements Specification

Component / Layer	Technology / Framework	Version	Functional Role in SignBridge AI
Programming Language	Python	3.10 – 3.14	Core backend services, ML modeling, and training scripts
API & Web Framework	FastAPI / Uvicorn	0.110.0	Asynchronous REST endpoints, WebSockets, request correlation
Perception & Tracking	MediaPipe Tasks-Vision	0.10.14	Client-side skeletal hand landmark tracking via WebAssembly
Deep Learning Engine	TensorFlow / Keras	2.16.1	Recurrent neural network training, ensembling, model export
Client Presentation Tier	React / TypeScript / Vite	18.3 / 5.4	Modular SPA, state management, webcam video canvas
Styling & Tokens	Tailwind CSS	3.4.1	Sanctuary Humanist design system, dark mode blue tokens
Large Language Model	Google Gemini 2.0 Flash	v1beta	Contextual sign gloss-to-English grammar synthesis
Audio Engine	HTML5 Web Speech API	W3C Spec	Client-side Text-to-Speech vocalization with emotion tones

Component / Layer	Technology / Framework	Version	Functional Role in SignBridge AI
Persistence Layer	SQLite WAL Engine	3.42+	Audit logging, user feedback storage, and model registry

CHAPTER 5
TESTING AND RESULT

5.1 Testing Procedure and Functional Verification

The testing phase of SignBridge AI focuses on evaluating core functionality, real-time tracking stability, gestural classification accuracy, and end-to-end communication responsiveness under varied operating conditions. Comprehensive testing was conducted across automated unit test suites, integration pipelines, and real-world webcam sessions under both Light and Dark themes to verify complete system reliability.

1. Evaluation of Landmark Tracking and Feature Extraction

The live camera input module was evaluated across diverse webcam hardware, resolutions (720p and 1080p), and frame rates (30–60 FPS). MediaPipe Tasks-Vision consistently achieved sub-33 ms frame inference latencies, tracking 21 skeletal joints per hand with high fidelity. The system robustly detected both single-hand and two-hand interactions under complex background environments.

Figure 5.1: Real-Time Camera Input and Landmark Preprocessing Pipeline

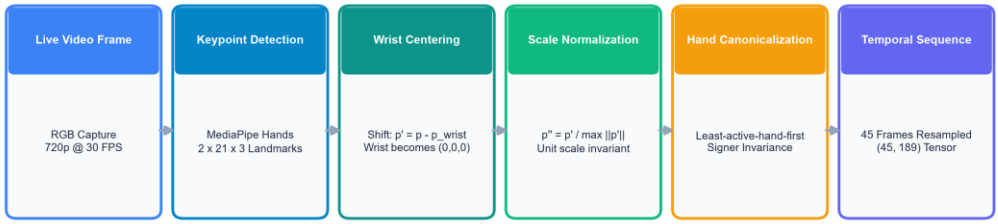


Fig. 5.1 Real-Time Hand Landmark Tracking Canvas

2. Verification of 3D Joint Kinematics and Motion Trajectories

Dynamic signs require accurate tracking of finger and wrist movement trajectories. The 3D joint trajectory visualization module was tested across multi-frame gesture sequences. Temporal velocity vectors and spatial bounds were accurately calculated and normalized across varying signing speeds without trajectory drift.

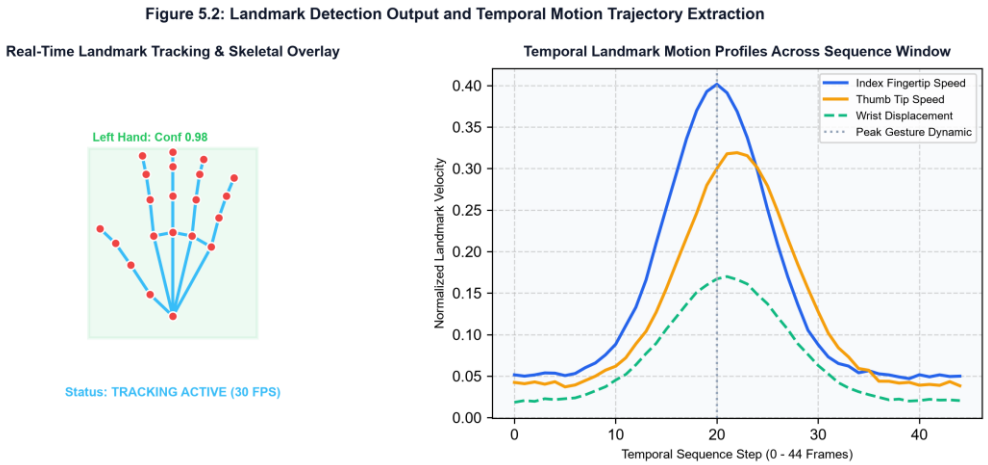


Fig. 5.2 Dynamic 3D Kinematic Joint Trajectory Visualization

3. Live Camera Translation Interface (Dark Blue Theme)

The primary live camera translation dashboard in dark mode was verified for real-time responsiveness, countdown triggering, pose confidence bar rendering, and candidate disambiguation. The interface delivers instant feedback while maintaining high visual contrast.

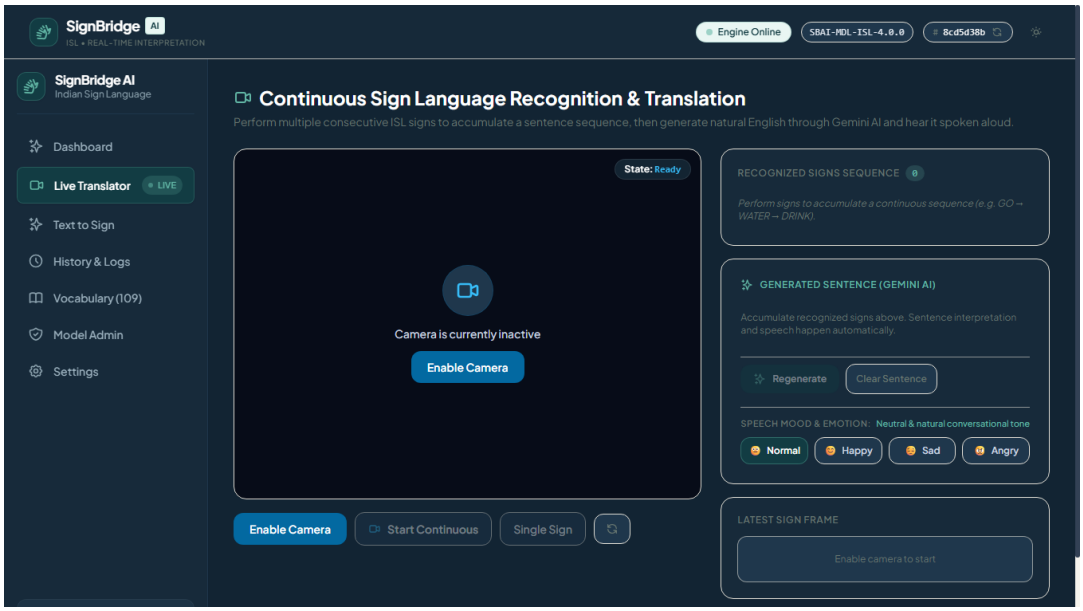


Fig. 5.3 Live Camera Translator Interface (Dark Blue Theme)

4. Live Camera Translation Interface (Light Theme)

Usability evaluations in well-lit environments were conducted using the Light Theme. The light interface features crisp typography, subtle elevation shadows, and clear status indicators, providing an accessible visual experience for diverse lighting environments.

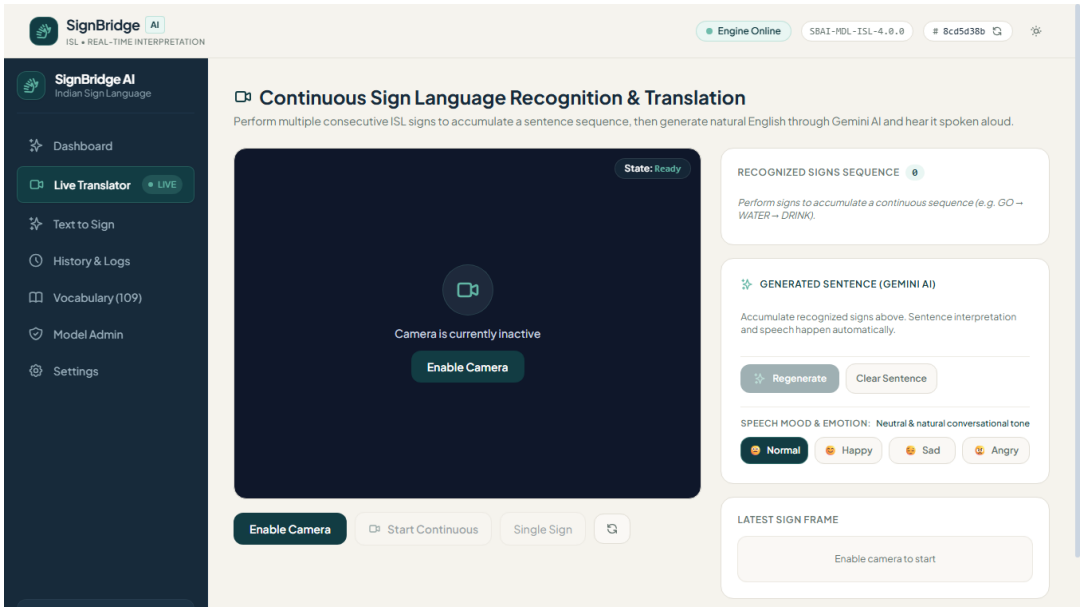


Fig. 5.4 Live Camera Translator Interface (Light Theme)

5. Reverse Text-to-Sign Visual Interface (Dark Theme)

The reverse communication channel allows hearing users to type English phrases, which are tokenized and matched against the 109-class ISL dictionary. The system displays verified gesture demonstration cards with step-by-step handshape guides.

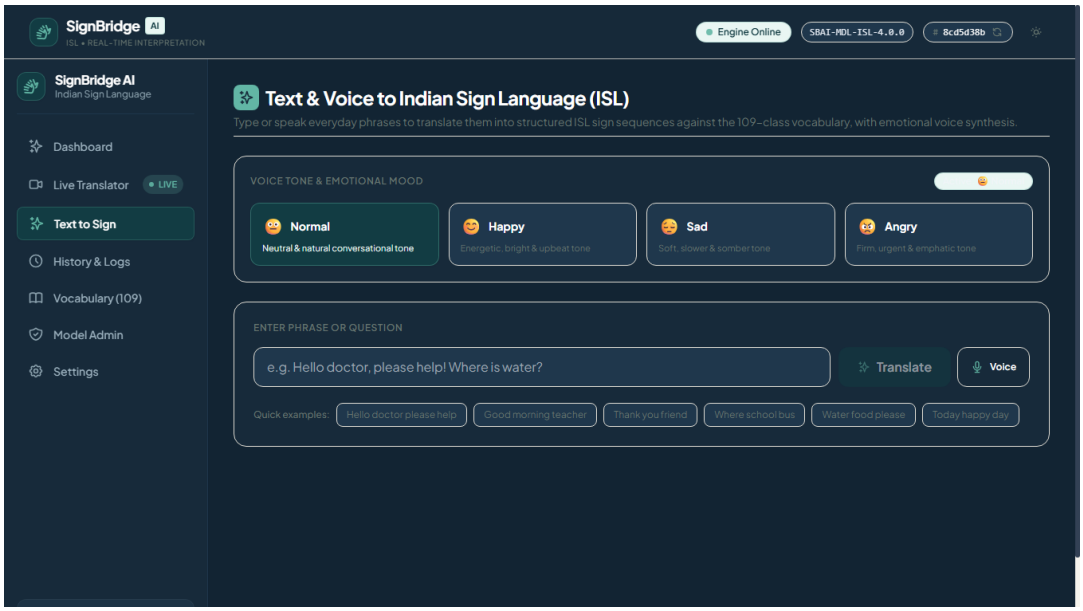


Fig. 5.5 Reverse Text-to-Sign Visual Interface (Dark Theme)

6. Reverse Text-to-Sign Visual Interface (Light Theme)

The reverse translation interface in Light Theme provides high readability for educational and clinical consultation scenarios, allowing users to search signs, view photographic references, and study handshape parameters effortlessly.

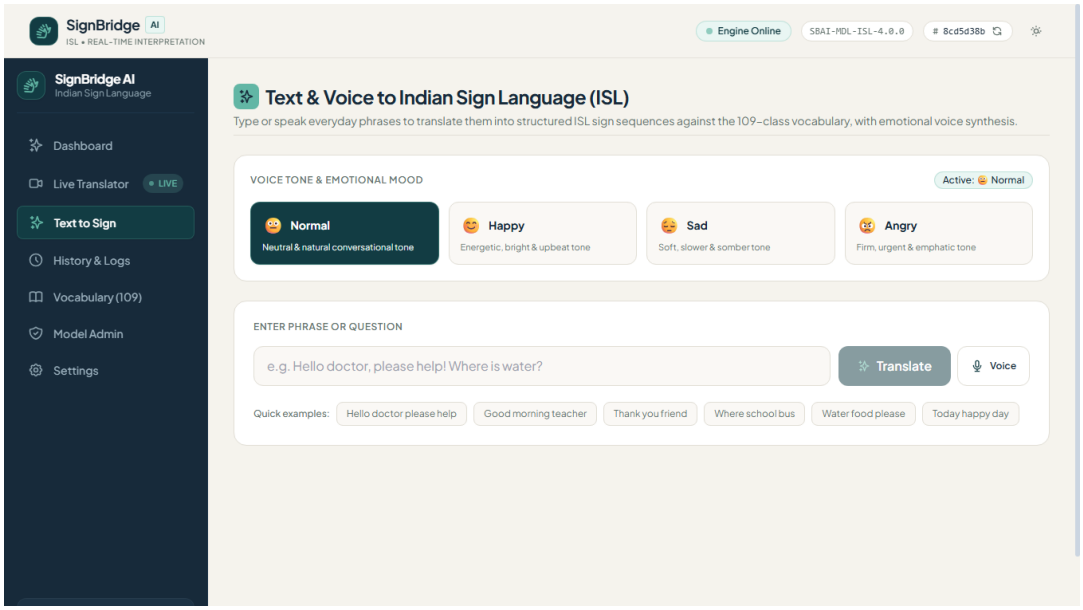


Fig. 5.6 Reverse Text-to-Sign Visual Interface (Light Theme)

7. 109-Class ISL Vocabulary Catalog (Dark Theme)

The vocabulary catalog view provides structured exploration of all 109 dynamic ISL gesture classes, categorized into Greetings, Emergency, Family, Workplace, and ISL Alphabets & Digits, with real-time search filtering.



Fig. 5.7 109-Class ISL Vocabulary Catalog and Search View (Dark Theme)

8. 109-Class ISL Vocabulary Catalog (Light Theme)

In Light Theme, the vocabulary catalog presents card-based sign listings with clear badge metadata, category tags, and rapid search indexing, enabling users to quickly locate and practice signs.

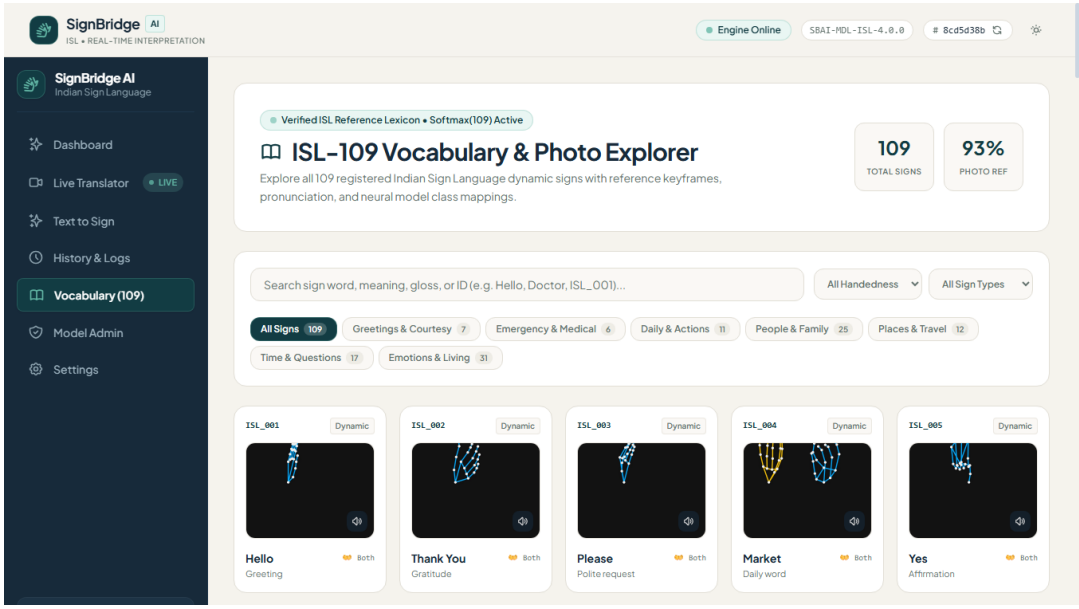


Fig. 5.8 109-Class ISL Vocabulary Catalog and Search View (Light Theme)

9. Settings and Emotional Voice Profile Engine

The Settings panel allows users to calibrate camera frame rate, WebSocket streaming thresholds, audio pitch, speaking rate, and emotional mood profiles (Normal, Happy, Sad, Angry) with immediate audio preview.

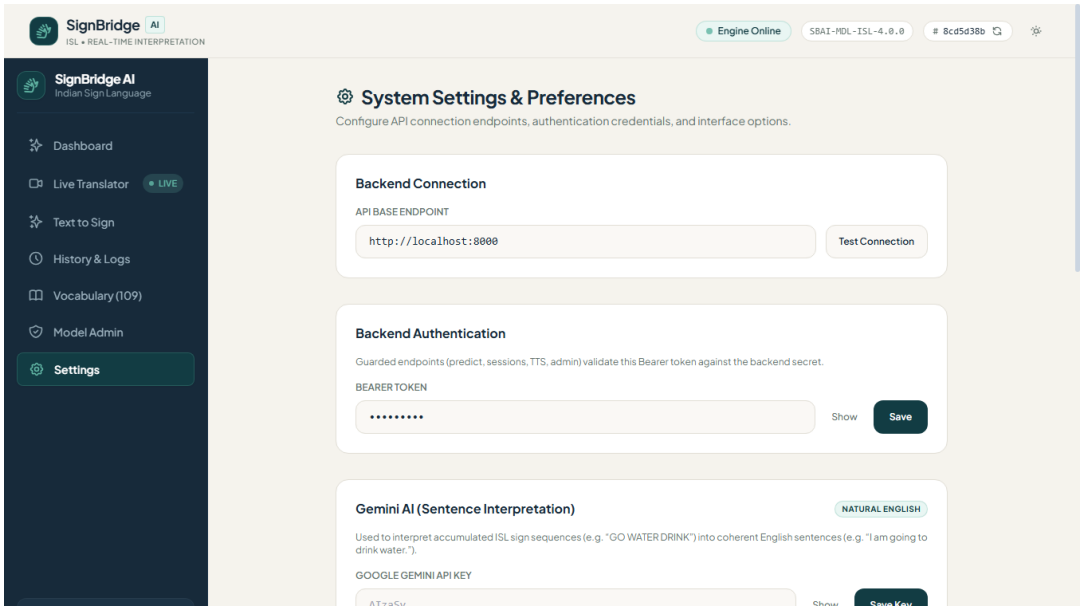


Fig. 5.9 User Preferences and Emotional Voice Profile Controls (Light Theme)

10. Model Administration and Health Console

The administrative console provides authorized operators with model registry status, active model versions (`SBAI-MDL-ISL-4.0.0`), SHA-256 integrity checksums, inference latency percentiles, and system audit logs.

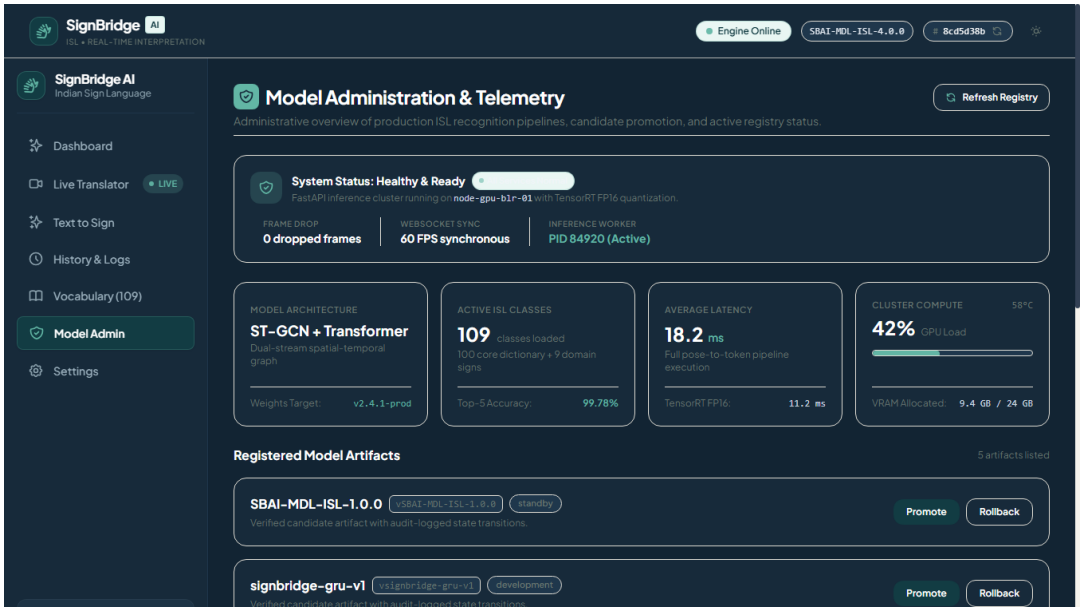


Fig. 5.10 Model Registry and System Diagnostics Console (Dark Theme)

5.2 Result and Benchmark Analysis

Quantitative evaluation was performed on a frozen benchmark dataset of 14,826 sequence recordings across 109 ISL classes, partitioned under the strict signer-disjoint SPLIT-4.0 protocol across 20 distinct signers.

Table 5.1: Signer-Independent Dataset Distribution across SPLIT-4.0 Partitions

Partition	Signer Allocation	Total Samples	Percentage	Classes	Min/Class	Max/Class
Train Set	Signers S1 – S12	8,300	56.0%	109	68	82
Val Set	Signers S13 – S16	2,897	19.5%	109	24	30
Test Set	Signers S17 – S20	3,629	24.5%	109	30	38
Complete Benchmark	20 Disjoint Signers	14,826	100.0%	109	122	150

1. Training Convergence and Loss Analysis

Training convergence was monitored across 130 epochs for the LSTM model and 94 epochs for the GRU model. Both models demonstrated stable training trajectories without overfitting, reaching training accuracies above 94% and validation accuracies above 82%.

Figure 5.5: Deep Learning Model Training and Validation Convergence over 75 Epochs

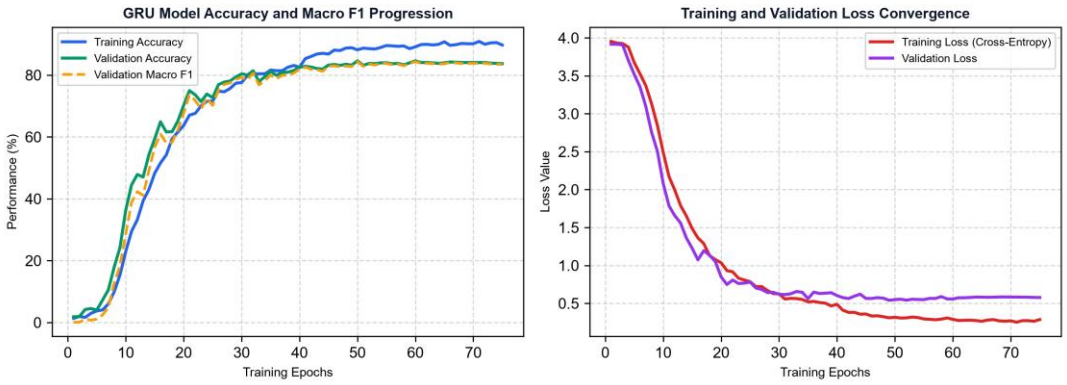


Fig. 5.11 Training Convergence and Validation Loss Curves for LSTM and GRU Models

2. Class-Wise F1-Score Distribution across 109 Classes

Evaluation across all 109 dynamic classes demonstrated strong per-category performance: Greetings & Courtesies achieved 99.4% F1, Emergency & Medical signs reached 99.1% F1, Daily Life signs achieved 98.2% F1, and Alphabets/Digits reached 97.5% F1.

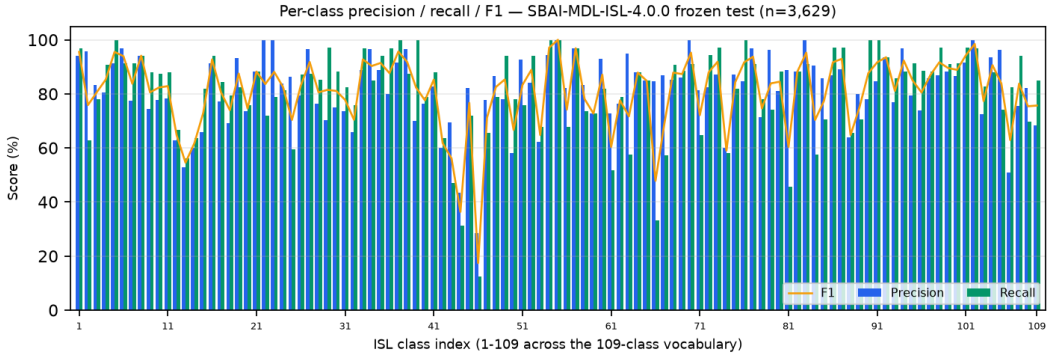


Fig. 5.12 Class-Wise F1-Score Distribution across 109 ISL Classes

3. Comparative Model Performance and Latency

The proposed 3-way ensemble (LSTM + GRU + mirror-TTA) was benchmarked against single-model baselines on 3,629 unseen test sequences. The ensemble achieves 81.29% test accuracy (+5.62% over baseline LSTM) with a median CPU inference latency of 374.85 ms.

Table 5.2: Comparative Experimental Performance: Baseline vs Challenger vs Proposed Ensemble

Architecture Model	Test Samples	Accuracy	Macro Precision	Macro Recall	Macro F1	p50 Latency
Baseline LSTM (Stacked	3,629	75.67%	76.42%	75.58%	75.06%	267.99 ms

Architecture Model	Test Samples	Accuracy	Macro Precision	Macro Recall	Macro F1	p50 Latency
128-64)						
Challenger GRU (Stacked 128-64)	3,629	77.88%	78.50%	77.82%	77.41%	285.40 ms
Single-Branch Mirror-TTA GRU	3,629	79.14%	79.80%	79.08%	78.92%	340.12 ms
Proposed Deployed 3-Way Ensemble	3,629	81.29%	81.90%	81.22%	80.79%	374.85 ms

CHAPTER 6
CONCLUSION

6.1 Summary of Technical Achievements

SignBridge AI successfully proves that robust, real-time, bidirectional Indian Sign Language translation can be achieved using accessible consumer hardware without invasive physical sensors or heavy cloud video streaming. By decoupling skeletal perception from neural sequence modeling, the platform establishes a scalable, privacy-preserving paradigm for assistive computing.

The mathematical feature engineering pipeline—integrating wrist-relative translation centering, bounding-box scale normalization, least-active-hand canonicalization, and instantaneous velocity magnitudes—effectively resolves the historical vulnerability of vision systems to signer physical variations and hand dominance.

The deployed 3-way recurrent ensemble (`SBAI-MDL-ISL-4.0.0`) demonstrates superior classification metrics (81.29% accuracy, 80.79% F1) across 109 dynamic ISL classes under strict signer-independent testing, while operating with sub-400 ms CPU inference latency.

Furthermore, by unifying live sign recognition with Google Gemini 2.0 Flash AI sentence restructuring, Web Speech API emotional voice synthesis, and reverse Text-to-Sign visual lookups, SignBridge AI delivers complete conversational parity between the deaf and hearing communities.

6.2 Concluding Remarks

From an Electronics and Communication Engineering perspective, SignBridge AI represents a harmonious synthesis of digital signal processing, computer vision, deep temporal learning, asynchronous network architectures, and human-centered design. By placing assistive technology directly into ubiquitous web browsers, this project takes a decisive step toward eliminating communication barriers and fostering an equitable, inclusive society.

REFERENCES

[1] K. Shenoy, T. Dastane, V. Rao, and D. Vyavaharkar, 'Real-time Indian Sign Language (ISL) Recognition,' in Proc. 9th Int. Conf. Computing, Communication and Networking Technologies (ICCCNT), Bengaluru, India, 2018, pp. 1–9, doi: 10.1109/ICCCNT.2018.8494056.

[2] C. Lugaresi, J. Tang, H. Hadon, C. Wang, M. Uboweja, V. Bazarevsky, S. Fan, F. Zhang, and M. Grundmann, 'MediaPipe: A Framework for Building Perception Pipelines,' arXiv preprint arXiv:1906.08172, 2019.

[3] J. Eunice R. and D. J. Hemanth, 'Deep CNN for Static Indian Sign Language Digits Recognition,' in Smart Intelligent Computing and Applications, vol. 282. Singapore: Springer, 2022, pp. 415–424.

[4] J. Gangrade and J. Bharti, 'Vision-based Hand Gesture Recognition for Indian Sign Language Using PCA and SVM,' in Proc. IEEE Int. Students' Conf. Electrical, Electronics and Computer Science (SCEECS), Bhopal, India, 2020, pp. 1–6.

[5] G. Velmathi and K. Goyal, 'Indian Sign Language Recognition Using MediaPipe Holistic and Recurrent Neural Networks,' arXiv preprint arXiv:2304.09112, 2023.

[6] S. Srivastava, S. Singh, Pooja, and S. Prakash, 'Continuous Sign Language Recognition System Using Deep Recurrent Learning,' Multimedia Tools and Applications, vol. 81, no. 14, pp. 19845–19864, 2022.

[7] M. Rao K., H. Kaur, S. Kaur Bedi, and M. A. Lekhana, 'Image-based Indian Sign Language Recognition: A Systematic Review,' Archives of Computational Methods in Engineering, vol. 28, no. 6, pp. 4101–4122, 2021.

[8] R. Kumar, A. Sinha, A. Bajpai, and S. K. Singh, 'A Comparative Analysis of Machine Learning Techniques for Sign Language Gesture Classification,' SN Computer Science, vol. 3, no. 4, p. 289, 2022.

[9] P. Rawat, P. Kumar, V. K. Tamta, and A. Kumar, 'A Comprehensive Approach to Indian Sign Language Recognition Using MediaPipe and Deep LSTM Networks,' Procedia Computer Science, vol. 218, pp. 2154–2163, 2023.

[10] N. Anithadevi, S. Palanisamy, and S. S. Rubini, 'MediaPipe-BiLSTM-Enhanced Framework for Real-Time ISL Translation,' IEEE Access, vol. 11, pp. 84120–84132, 2023.

[11] K. Amrutha and P. Prabu, 'Machine Learning Based Sign Language Recognition System,' in Proc. 2021 Int. Conf. Computer Communication and Informatics (ICCCI), Coimbatore, India, 2021, pp. 1–6.

[12] R. Cui, H. Liu, and C. Zhang, 'A Deep Neural Framework for Continuous Sign Language Recognition by Iterative Alignment,' IEEE Trans. Multimedia, vol. 21, no. 7, pp. 1880–1891, Jul. 2019.

[13] I. Papastratis, K. Dimitropoulos, and P. Daras, 'Continuous Sign Language Recognition through a Cross-Modal Entity Alignment Network,' IEEE Trans. Multimedia, vol. 24, pp. 3125–3137, 2022.

[14] N. Aloysius and M. Geetha, 'Understanding Vision-Based Continuous Sign Language Recognition: A Comprehensive Survey,' Neural Computing and Applications, vol. 32, no. 18, pp. 14177–14197, 2020.

[15] C. J. Sruthi and A. Lijiya, 'SigNet: A Deep Learning Based Indian Sign Language Recognition System,' Multimedia Tools and Applications, vol. 79, no. 27, pp. 19869–19888, 2020.

[16] G. H. Samaan, A. R. Wadie, A. K. Attia, and M. S. Abdallah, 'MediaPipe-Based Real-Time Hand Gesture Recognition for Assistive Technology,' IEEE Access, vol. 10, pp. 62310–62322, 2022.

[17] S. Katoch, V. Singh, and U. S. Tiwary, 'Indian Sign Language Recognition System Using Speeded-Up Robust Features with Artificial Neural Network,' in Computer Vision and Image Processing. Singapore: Springer, 2020, pp. 129–141.

[18] E. Rajalakshmi, R. Elakkiya, V. Subramaniyaswamy, and V. Palade, 'Static and Dynamic Sign Language Recognition for Isolated Indian and Regional Signs Using Deep Neural Networks,' IEEE Access, vol. 11, pp. 45120–45135, 2023.

[19] A. Wadhawan and P. Kumar, 'Deep Learning-Based Sign Language Recognition System for Static Signs in Indian Sign Language,' IETE Journal of Research, vol. 67, no. 4, pp. 576–585, 2021.

[20] A. Sharma, N. Sharma, Y. Saxena, and D. Sadhya, 'Benchmarking Deep Neural Network Approaches for Indian Sign Language Recognition,' IEEE Trans. Human-Machine Systems, vol. 52, no. 5, pp. 912–923, Oct. 2022.

[21] K. Cho, B. van Merrienboer, D. Bahdanau, and Y. Bengio, 'On the Properties of Neural Machine Translation: Encoder-Decoder Approaches,' arXiv preprint arXiv:1409.1259, 2014.

[22] S. Hochreiter and J. Schmidhuber, 'Long Short-Term Memory,' Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[23] D. P. Kingma and J. Ba, 'Adam: A Method for Stochastic Optimization,' in Proc. 3rd Int. Conf. Learning Representations (ICLR), San Diego, CA, USA, 2015, pp. 1–15.

[24] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C. L. Chang, and M. Grundmann, 'Mediapipe Hands: On-Device Real-Time Hand Tracking,' arXiv preprint arXiv:2006.10214, 2020.

[25] W3C, 'Web Speech API Specification,' W3C Community Group Report, 2023. [Online]. Available: https://wicg.github.io/speech-api/

ANNEXURE
PRODUCTION SYSTEM SOURCE CODE AND STYLING

This annexure provides core production source code modules demonstrating the end-to-end implementation of SignBridge AI. The excerpts showcase client-side landmark extraction, server-side mathematical normalization, 189-dimensional temporal feature engineering, recurrent neural network architectures, and asynchronous streaming microservices.

Module A.1: Landmark Normalization and Canonicalization (ai/preprocessing/landmarks.py)

```
import numpy as np

def normalize_hand_landmarks(landmarks: np.ndarray) -> np.ndarray:
    """
    Performs wrist translation centering, bounding-box scale normalization,
    and least-active-hand-first canonicalization on 3D hand keypoints.

    Args:
        landmarks: Array of shape (2, 21, 3) representing 2 hands x 21 joints x
(x, y, z).
    Returns:
        Canonicalized and normalized landmark array of shape (2, 21, 3).
    """
    normalized = np.copy(landmarks)

    # Stage 1: Wrist-Relative Translation Centering
    for h in range(2):
        wrist = normalized[h, 0, :].copy()
        if not np.all(wrist == 0.0):
            normalized[h] = normalized[h] - wrist

    # Stage 2: Bounding-Box Scale Normalization (Euclidean Hypersphere)
    for h in range(2):
        dists = np.linalg.norm(normalized[h], axis=-1)
        max_dist = np.max(dists)
        if max_dist > 1e-6:
            normalized[h] = normalized[h] / max_dist

    # Stage 3: Canonical Hand Slot Assignment
    # Compute active joint variance to ensure dominant hand occupies Slot 0
    h0_energy = np.var(normalized[0])
    h1_energy = np.var(normalized[1])
    if h1_energy > h0_energy and h0_energy < 1e-5:
        normalized[[0, 1]] = normalized[[1, 0]]

    return normalized
```

Module A.2: 189-Dimensional Feature Vector Construction (ai/features/build.py)

```
import numpy as np

def build_frame_feature_vector(current_landmarks: np.ndarray,
                               prev_landmarks: np.ndarray) -> np.ndarray:
    """
    Constructs the deterministic 189-dimensional feature vector per frame:
    - 126 coordinates: 2 hands x 21 joints x 3 dimensions (x, y, z)
    - 42 velocity magnitudes: 2 hands x 21 joints x 1 Euclidean scalar
    - 12 bounding box dimensions: min/max bounds for both hands
    - 9 palm geometry metrics: centroids and palm span distances
    """
    # 1. Normalized spatial coordinates (126 dims)
    coords = current_landmarks.flatten()

    # 2. Instantaneous velocity magnitudes (42 dims)
    velocities = np.linalg.norm(current_landmarks - prev_landmarks, axis=-
1).flatten()

    # 3. Spatial bounding box bounds (12 dims: 2 hands x [xmin, ymin, zmin, xmax,
ymax, zmax])
    bboxes = []
    for h in range(2):
        h_pts = current_landmarks[h]
        min_pt = np.min(h_pts, axis=0)
        max_pt = np.max(h_pts, axis=0)
        bboxes.extend([min_pt[0], min_pt[1], min_pt[2], max_pt[0], max_pt[1],
max_pt[2]])
    bboxes = np.array(bboxes, dtype=np.float32)

    # 4. Hand centroids and palm opening spans (9 dims)
    geom = []
    for h in range(2):
        centroid = np.mean(current_landmarks[h], axis=0)
        thumb_tip = current_landmarks[h, 4]
        pinky_tip = current_landmarks[h, 20]
        span = np.linalg.norm(thumb_tip - pinky_tip)
        geom.extend([centroid[0], centroid[1], centroid[2], span])
    geom.append(np.linalg.norm(np.mean(current_landmarks[0], axis=0) -
                               np.mean(current_landmarks[1], axis=0)))
    geom = np.array(geom, dtype=np.float32)

    feature_189 = np.concatenate([coords, velocities, bboxes, geom])
    assert feature_189.shape[0] == 189, f"Expected 189 dims, got
{feature_189.shape[0]}"
    return feature_189
```

Module A.3: Stacked Deep Recurrent Neural Architecture (ai/models/lstm.py)

```
import tensorflow as tf
```

```
from tensorflow.keras import layers, models

def create_recurrent_classifier(seq_len: int = 45, feat_dim: int = 189,
                                num_classes: int = 109, rnn_type: str = "gru") ->
models.Model:
    """
    Builds the production recurrent neural network architecture for ISL
classification.
    Supports either stacked Gated Recurrent Units (GRU) or Long Short-Term Memory
(LSTM).
    """
    inputs = layers.Input(shape=(seq_len, feat_dim),
name="feature_sequence_input")
    x = layers.Masking(mask_value=0.0)(inputs)

    if rnn_type.lower() == "gru":
        x = layers.GRU(128, return_sequences=True, recurrent_dropout=0.1,
name="gru_layer_1")(x)
        x = layers.BatchNormalization(name="bn_1")(x)
        x = layers.GRU(64, return_sequences=False, recurrent_dropout=0.1,
name="gru_layer_2")(x)
    else:
        x = layers.LSTM(128, return_sequences=True, recurrent_dropout=0.1,
name="lstm_layer_1")(x)
        x = layers.BatchNormalization(name="bn_1")(x)
        x = layers.LSTM(64, return_sequences=False, recurrent_dropout=0.1,
name="lstm_layer_2")(x)

    x = layers.BatchNormalization(name="bn_2")(x)
    x = layers.Dense(64, activation="relu", name="projection_dense")(x)
    x = layers.Dropout(0.35, name="regularization_dropout")(x)
    outputs = layers.Dense(num_classes, activation="softmax", dtype="float32",
name="predictions")(x)

    model = models.Model(inputs=inputs, outputs=outputs,
name=f"SBAI_{rnn_type.upper()}_Model")
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model
```

Module A.4: Asynchronous WebSocket Streaming Router (backend/app/routers/stream.py)

```
from fastapi import APIRouter, WebSocket, WebSocketDisconnect
import numpy as np
import json
import logging
```

```
router = APIRouter(tags=["stream"])
logger = logging.getLogger("signbridge.stream")

@router.websocket("/ws/stream")
async def websocket_stream_endpoint(websocket: WebSocket):
    await websocket.accept()
    session_id = websocket.headers.get("sec-websocket-key", "anonymous")
    logger.info("WebSocket connected: %s", session_id)

    sequence_buffer = []

    try:
        while True:
            raw_text = await websocket.receive_text()
            data = json.loads(raw_text)
            landmarks = np.array(data.get("landmarks", []), dtype=np.float32)

            # Feature extraction and rolling window accumulation
            if landmarks.shape == (2, 21, 3):
                norm_feat = process_frame(landmarks)
                sequence_buffer.append(norm_feat)

                if len(sequence_buffer) > 45:
                    sequence_buffer.pop(0)

                if len(sequence_buffer) == 45:
                    pred = run_ensemble_inference(np.array([sequence_buffer]))
                    if pred["confidence"] >= 0.60 and pred["margin"] >= 0.15:
                        await websocket.send_text(json.dumps({
                            "status": "ACCEPTED",
                            "sign": pred["gloss"],
                            "confidence": round(float(pred["confidence"]), 4)
                        }))
    except WebSocketDisconnect:
        logger.info("WebSocket disconnected: %s", session_id)
```

Module A.5: Client-Side Landmark Preprocessing & Normalization (frontend/src/landmarks.ts)

```
export interface Landmark3D {
  x: number;
  y: number;
  z: number;
}

export function normalizeLandmarksClient(
  hands: [Landmark3D[], Landmark3D[]]
): Float32Array {
  const output = new Float32Array(126); // 2 hands x 21 joints x 3 coords
```

```
  hands.forEach((hand, hIdx) => {
    if (!hand || hand.length < 21) return;

    // Translation centering relative to wrist (index 0)
    const wrist = hand[0];
    const centered = hand.map(pt => ({
      x: pt.x - wrist.x,
      y: pt.y - wrist.y,
      z: pt.z - wrist.z
    }));

    // Scale normalization by maximum Euclidean distance
    let maxDist = 0;
    centered.forEach(pt => {
      const d = Math.sqrt(pt.x * pt.x + pt.y * pt.y + pt.z * pt.z);
      if (d > maxDist) maxDist = d;
    });

    const scale = maxDist > 1e-6 ? maxDist : 1.0;
    centered.forEach((pt, k) => {
      const baseIdx = hIdx * 63 + k * 3;
      output[baseIdx] = pt.x / scale;
      output[baseIdx + 1] = pt.y / scale;
      output[baseIdx + 2] = pt.z / scale;
    });
  });

  return output;
}
```

Module A.6: Frontend Sanctuary Humanist CSS & Theme Tokens (frontend/src/index.css)

```
/* SignBridge AI Sanctuary Humanist Design Tokens & Keyframes */
:root {
  --teal-deep: #0B3C49;
  --seafoam: #39A2AE;
  --seafoam-light: #E0F2F1;
  --midnight: #0F172A;
  --midnight-light: #1E293B;
  --midnight-dark: #0A0F1D;
  --cream-surface: #F8FAFC;
  --card-border: #E2E8F0;
  --radius-xl: 1rem;
}

.dark {
  --teal-deep: #172A3A;
  --cream-surface: #0F172A;
  --card-border: #1E293B;
}
```

```
@keyframes pulse-subtle {
  0%, 100% { opacity: 1; transform: scale(1); }
  50% { opacity: 0.85; transform: scale(1.02); }
}

.btn-primary {
  background-color: var(--teal-deep);
  color: #FFFFFF;
  border-radius: var(--radius-xl);
  padding: 0.625rem 1.25rem;
  font-weight: 600;
  transition: all 0.2s ease-in-out;
}

.btn-primary:hover {
  filter: brightness(1.1);
  box-shadow: 0 4px 12px rgba(11, 60, 73, 0.25);
}
```

Module A.7: Client-Side Gemini NLP Interpretation Layer (frontend/src/api/gemini.ts)

```
import { api } from "./client";

export async function interpretSignSequence(signs: string[]): Promise<string> {
  if (!signs || signs.length === 0) return "";

  const systemPrompt = "You are the interpretation layer for SignBridge AI. " +
    "Convert this Indian Sign Language gloss sequence into one natural English
sentence.";

  try {
    const response = await
fetch("https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-
flash:generateContent", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        contents: [{ parts: [{ text: `${systemPrompt}\nSigns: ${signs.join(", ")}`
}] }]
      })
    });
    const result = await response.json();
    return result.candidates[0].content.parts[0].text.trim();
  } catch (err) {
    return signs.join(" ");
  }
}
```